

Notes

- What is **Authentication**?
 - It is just a way of identifying who someone is? ⇒ **Identification**.
- What is **Authorisation**?
 - It is a way of determining what access does someone have? ⇒ **Permission**.
- Without **Authentication** I can not buy a product on **Amazon**, even if i am **authenticated**, I cannot update or delete the product details.
- Most common **Authentication** .
 - Signup/Register/Create Account
 - Login.
- If **Email** and **Password** is correct only then **Login** .
- We have to match it with the data that is there in the database.
- Create a backend project now.
- `npm i mongoose express nodemon dotenv`

```
//server.js
const express=require("express")
const {connection}=require("./config/db")

const app=express()
app.use(express.json())

app.post("/register",async (req,res)=>{
  try{
    const user= new UserModel(req.body)
    await user.save()
  }catch(err){
    console.log(err)
  }
  res.send("Registered")
})

app.post("/login",async (req,res)=>{
  const {email,pass}=req.body
  try{
    const user=await UserModel.find({email,pass})
    if(user.length>0){
      res.send("Login Successful")
    } else {
      res.send("Login Failed")
    }
  }
})
```

```

    }
  } catch(err){
    console.log(err)
  }
})

app.listen(8080, async () => {
  try {
    await connection
  } catch(err) {
    console.log(err)
  }
  console.log("Running at 8080 Port")
})

```

```

//config/db.js

const mongoose = require("mongoose")

const connection = mongoose.connect("mongodb_url")

module.exports = {
  connection
}

```

```

//models/User.model.js

const userSchema = mongoose.Schema({
  email: String,
  pass: String,
  name: String,
  age: Number
})

const UserModel = mongoose.model("user", userSchema)

```

- Now we are able to `register` and `login`, now we have to make some `authenticated endpoints` as well.
 - `/data` endpoint.
 - `/userdetails` endpoint
- Cannot be done at `frontend` as always they are not going to use `API` endpoint using `FE`. `Thunderclient` or `Postman` can also be used
- What should be done here?

- **1st Way:** We can make them login at protected endpoints. (send email and password with request when they are accessing this endpoint)⇒Not the best way⇒Because this could be really annoying.⇒ Every time we have to go through the data and check it.
- **2nd Way:** Access Token can be an option. Give them a token when they are logged in.
- Only check for that token while visiting the restricted endpoint.

```
//send token while login
app.post("/login", async (req, res) => {
  const {email, pass} = req.body
  try {
    const user = await UserModel.find({email, pass})
    if (user.length > 0) {
      res.send({ "msg": "Login Successful", "token": "abc123" })
    } else {
      res.send("Login Failed")
    }
  } catch (err) {
    console.log(err)
  }
})

app.get("/data", (req, res) => {
  if (req.query.token === "abc123") {
    res.send("Loggen in and can access the data")
  } else {
    res.send("Not Loggen in, login first")
  }
})

app.get("/cart", (req, res) => {
  if (req.query.token === "abc123") {
    res.send("Loggen in and can access the cart")
  } else {
    res.send("Not Loggen in, login first")
  }
})
}
```

Lets us look at better way to do this

- There is something that we can use, which is unique, and verifiable as well.
- **JWT⇒JSON WEB TOKEN**
- It is made up of three different things.
 - Algorithm to encrypt the string.

- It can be decrypted.
- Payload as well, any data, which will also be encrypted in the string.
- A secret Key to decrypt it.

Install the JWT token

- `npm i jsonwebtoken`

```
const jwt=require("jsonwebtoken")

//while logging in
const token=jwt.sign({"course":"backend"}, "porwal")
```

- This token will be used in restricted routes now.

```
//while visiting /data or /userdetails
var token=req.headers.authorization
jwt.verify(token,"batman" (err,decoded )=>{
  if(err){
    res.send("Invalid Token")
    console.log(err)
  } else {
    res.send("Whatever you want can be sent over here")
  }
})
```

- We are not going to pass it in `queries` , we are going to take it from `headers` .